Recuperación Aumentada sobre Documentos PDF con Agente Conversacional

Cristopher Josua Marrasquin Gordon
Especialización en Inteligencia Artificial y Big Data — Institut TIC Barcelona

Abstract—Este trabajo presenta el diseño e implementación de un sistema de generación aumentada por recuperación (RAG) orientado al apoyo en el estudio a partir de documentos PDF. El sistema permite a los estudiantes interactuar de forma conversacional con sus propios materiales académicos, obteniendo respuestas contextualmente relevantes generadas a partir del contenido real de los documentos. La arquitectura integra una aplicación móvil en React Native, un backend asíncrono en FastAPI y una base de datos híbrida en Supabase que combina almacenamiento relacional, vectorial y de objetos. El procesamiento semántico se apoya en el modelo de embeddings *nomic-embed-text-v2-moe* con vectores de 512 dimensiones. La orquestación del agente se realiza mediante LangGraph con un patrón de tool calling, evolucionando desde una arquitectura inicial de grafo explícito con router hacia un agente conversacional más flexible. El sistema fue evaluado con tres modelos de lenguaje: Gemma 4 E4B, Gemma 4 31B y Claude Haiku.

1 Introducción

Estudiar a partir de documentos PDF es algo cotidiano para cualquier estudiante: apuntes de clase, libros de texto, artículos del profesor. El problema es que cuando surge una duda concreta, encontrar la respuesta dentro de un documento extenso es frustrante. Se puede buscar una palabra, pero no se puede preguntarle al documento.

Los modelos de lenguaje han demostrado que es posible mantener una conversación natural sobre cualquier tema. Sin embargo, estos modelos tienen un problema fundamental: su conocimiento está fijado en el momento del entrenamiento y no saben nada sobre los apuntes específicos de un curso. Si les preguntas sobre el temario de tu asignatura, inventan o generalizan.

La generación aumentada por recuperación, conocida como RAG, resuelve esto: en lugar de depender solo del conocimiento interno del modelo, el sistema busca primero los fragmentos relevantes dentro del documento del usuario y los usa como contexto para generar la respuesta. El resultado es un asistente que responde basándose en lo que realmente pone en tus apuntes.

Este trabajo presenta el diseño e implementación de un sistema de este tipo, accesible desde el móvil, donde el estudiante puede subir sus propios PDFs y hacer preguntas sobre ellos en lenguaje natural. Las principales contribuciones son: una arquitectura completa que integra recuperación semántica, un agente conversacional y respuestas en tiempo real; el análisis comparativo de dos diseños de orquestación con LangGraph; la gestión persistente del historial de conversación; y la evaluación del sistema con tres modelos de lenguaje distintos.

El artículo se organiza así: la sección 2 revisa los trabajos relacionados; la sección 3 describe la arquitectura; la sección 4 detalla el pipeline RAG y el agente; la sección 5 presenta la evaluación; y la sección 6 recoge las conclusiones.

2 Trabajo Relacionado

Generación aumentada por recuperación. Lewis et al. [1] introdujeron la arquitectura RAG combinando recuperación densa con generación secuencia a secuencia. Gao et al. [2] presentaron una clasificación de las distintas variantes del sistema, desde implementaciones básicas hasta arquitecturas más sofisticadas con recuperación iterativa y razonamiento encadenado, describiendo la evolución hacia sistemas con recuperación iterativa y razonamiento. El presente trabajo adopta una variante modular donde la lógica de recuperación es delegada a herramientas invocadas por el propio modelo.

LLMs en educación. Baidoo-Anu y Owusu Ansah [3] analizaron el potencial de los asistentes conversacionales para la tutoría personalizada. La mayoría de estas propuestas operan sobre el conocimiento general del modelo, sin anclar las respuestas en los materiales específicos del estudiante, limitación que el presente sistema aborda mediante recuperación semántica sobre los documentos propios del usuario.

Sistemas de tutoría inteligente. Bloom [4] demostró que la tutoría individualizada mejora significativamente el rendimiento académico. La integración de LLMs abre la posibilidad de construir tutores conversacionales capaces de razonar sobre documentos específicos del curso.

Bases de datos vectoriales. La extensión pgvector [6] integra capacidades de búsqueda vectorial directamente en PostgreSQL mediante el operador de distancia coseno <=>, eliminando la necesidad de servicios externos y simplificando la gestión de consistencia.

Agentes con tool calling. El patrón ReAct [5] propone intercalar razonamiento y acción en un ciclo itera-

tivo. LangGraph [7] extiende este patrón representando el flujo de ejecución como un grafo de estados dirigido, facilitando la implementación de lógica condicional y trazabilidad del proceso.

3 Arquitectura del Sistema

El sistema está dividido en tres capas bien diferenciadas: la interfaz de usuario, la lógica de procesamiento y el almacenamiento de datos. Cada capa tiene una responsabilidad clara y se comunica con las demás mediante APIs REST y Server-Sent Events (SSE) para el streaming en tiempo real.

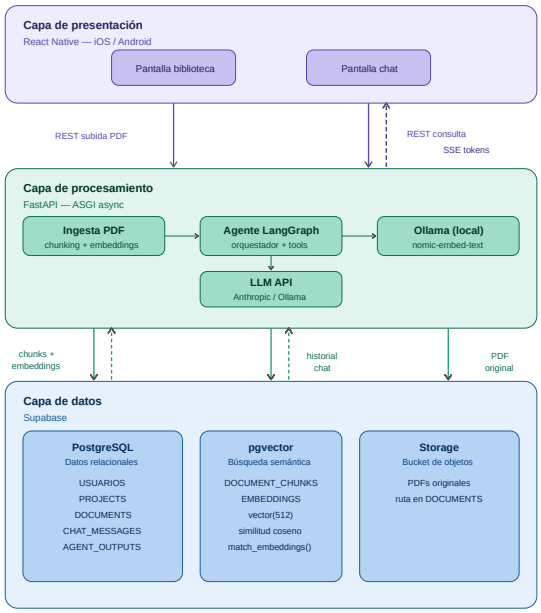


Figure 1: Arquitectura general del sistema en tres capas.

3.1 Aplicación Móvil (React Native)

La capa de presentación es una aplicación móvil desarrollada en React Native, lo que permite publicarla tanto en iOS como en Android desde un único proyecto. Está organizada en dos pantallas: la biblioteca, donde el usuario gestiona sus documentos PDF, y el chat, donde puede hacer preguntas sobre su contenido.

La respuesta del sistema llega token a token mediante una conexión SSE, de forma que el texto aparece progresivamente en pantalla, igual que cuando escribes un mensaje. La aplicación también ofrece accesos directos para generar un resumen o unas flashcards del documento con un solo toque, sin necesidad de escribir nada.

3.2 Backend (FastAPI)

La capa de procesamiento está implementada en FastAPI, framework asíncrono para Python basado en ASGI. Su compatibilidad nativa con `async/await` permite gestionar múltiples sesiones concurrentes sin bloqueo.

El backend expone dos grupos de endpoints: el primero gestiona la ingesta de documentos (recepción del PDF, pipeline de fragmentación y embeddings, subida a Supabase Storage y almacenamiento en base de datos); el segundo gestiona la interacción conversacional (ejecución asíncrona del agente LangGraph y transmisión de la respuesta mediante streaming SSE).

3.3 Base de Datos (Supabase)

La capa de datos se apoya en Supabase, que combina una base de datos PostgreSQL con almacenamiento de archivos y la extensión pgvector. Esta extensión permite guardar vectores directamente en la base de datos y hacer búsquedas por similitud semántica sin necesidad de ningún servicio externo, lo que simplifica enormemente la arquitectura y mantiene todos los datos en un único lugar.

El esquema está organizado en varias tablas con responsabilidades bien diferenciadas.

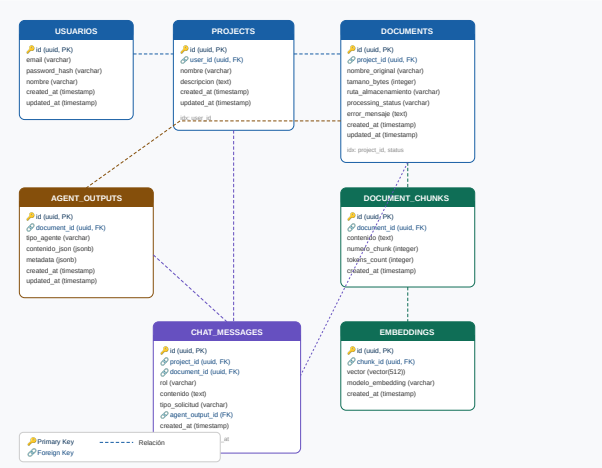


Figure 2: Esquema de base de datos en Supabase.

La tabla `documents` almacena los metadatos de cada PDF y su ruta en Supabase Storage, donde se guarda el archivo original. La tabla `document_chunks` almacena cada fragmento de texto con referencia al documento y su posición. La tabla `embeddings` guarda el vector de 512 dimensiones de cada chunk en una columna `vector(512)`.

Separar los chunks y los embeddings en tablas distintas tiene una ventaja práctica: si en el futuro se quiere cambiar el modelo de embeddings, basta con regenerar esa tabla sin tocar el texto original ni volver a procesar el PDF.

La tabla `chat_messages` guarda el historial de la conversación, con una columna `rol` que indica si

el mensaje es del usuario o del asistente, y se ordena por fecha de inserción para reconstruir el hilo correctamente.

La función SQL `match_embeddings` es la pieza central de la búsqueda: recibe el vector de la consulta, lo compara con todos los embeddings del documento activo y devuelve los k fragmentos más similares. Además aplica un umbral mínimo de similitud de 0.5, de forma que si la pregunta no tiene relación con el contenido del documento, el sistema no devuelve fragmentos irrelevantes al modelo.

4 Pipeline RAG y Agente

4.1 Ingesta y Procesamiento

El pipeline de ingesta comprende tres subetapas. La **extracción de texto** recorre el PDF página a página; los elementos no textuales son omitidos, lo que constituye una limitación en documentos con alto contenido visual.

La **fragmentación** divide el texto en fragmentos de 500 caracteres con un solapamiento de 100 caracteres entre fragmentos consecutivos. El solapamiento evita que un concepto que ocupa varias líneas quede cortado a la mitad, perdiendo parte de su significado en la búsqueda.

La **generación de embeddings** transforma cada fragmento en un vector de 512 dimensiones. Los resultados se almacenan en las tablas `document_chunks` y `embeddings` de Supabase.

4.2 Modelo de Embeddings

El modelo `nomic-embed-text-v2-moe` [12] es un modelo de arquitectura Mixture of Experts (MoE) ejecutado localmente a través de Ollama. La arquitectura MoE activa selectivamente distintos subconjuntos de parámetros según el token procesado, manteniendo alta capacidad expresiva con un coste computacional inferior a los modelos densos equivalentes. Además, permite configurar la dimensión de los vectores generados, lo que da flexibilidad para ajustar el balance entre precisión semántica y consumo de almacenamiento según las necesidades del sistema.

Un aspecto crítico es que el mismo modelo se utiliza tanto en la ingesta como en la recuperación, garantizando que ambos vectores habiten el mismo espacio semántico y que la similitud coseno sea una medida significativa de relevancia. La ejecución local elimina la dependencia de APIs externas durante el procesamiento.

4.3 Recuperación Semántica

Cuando el usuario escribe una pregunta, el sistema la convierte primero en un vector usando el mismo modelo de embeddings que se utilizó durante la ingesta. Esto es importante: al usar el mismo modelo, la pregunta y los fragmentos del documento viven en el mismo espacio semántico, por lo que la comparación tiene sentido.

Con ese vector, se ejecuta la función `match_embeddings` en Supabase, que realiza la búsqueda en un único paso SQL: compara el vector de la pregunta con todos los embeddings del documento, filtra los que superan el umbral mínimo de similitud y devuelve los k=5 fragmentos más relevantes.

La búsqueda por similitud coseno tiene una ventaja clave frente a la búsqueda por palabras clave: no importa si el usuario usa términos distintos a los del documento. Si el concepto es el mismo, la similitud semántica lo detecta igualmente.

4.4 Evolución del Grafo LangGraph

El diseño del agente evolucionó desde una arquitectura de grafo explícito con router hacia un agente con tool calling, motivado por limitaciones concretas encontradas durante la implementación.

4.4.1 Primera versión: grafo con router

La primera versión organizaba el flujo en siete nodos: `cargar_historial`, `router`, `retriever`, `resumen`, `corrector`, `flashcards` y `explicacion`. El estado que se pasaba entre nodos era un objeto con campos fijos para cada tipo de tarea: `query`, `document_id`, `tipo_agente`, `contexto`, `respuesta` e `historial`.

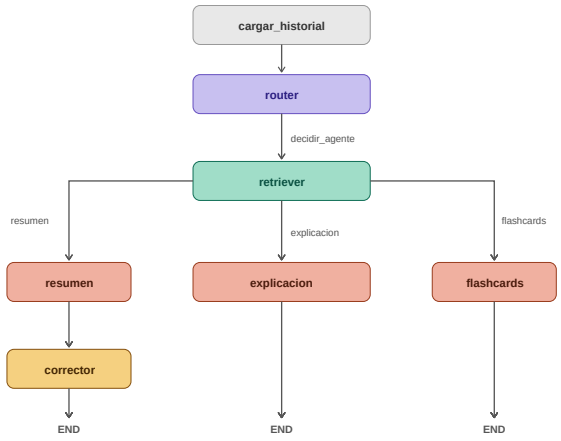


Figure 3: Primera versión: grafo con router explícito y siete nodos.

Esta arquitectura presentó varias limitaciones. El router debía clasificar la intención del usuario antes de hacer nada, lo que funcionaba bien para consultas claras pero fallaba con preguntas ambiguas o mixtas donde la categoría no estaba clara. Todos los caminos pasa-

ban obligatoriamente por `retriever`, incluso para operaciones como el resumen o las flashcards que no siempre necesitan buscar fragmentos específicos, añadiendo pasos innecesarios. Añadir una funcionalidad nueva obligaba a tocar el objeto de estado global, ya que tenía campos fijos pensados para las tareas existentes. Por último, cargar el historial como primer nodo del grafo mezclaba una operación de base de datos con la lógica del agente, cuando en realidad son dos responsabilidades distintas que deberían estar separadas.

4.4.2 Segunda versión: agente con tool calling

La segunda versión simplifica la arquitectura a dos nodos: `orquestador` y `tools`. El estado pasa a ser una lista de mensajes, alineándose con el paradigma conversacional estándar de LangChain.

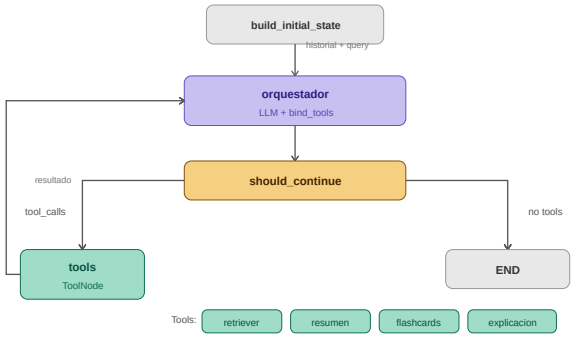


Figure 4: Segunda versión: agente con tool calling y ciclo ReAct.

El nodo `orquestador` invoca el LLM con cuatro herramientas vinculadas mediante `bind_tools`: `retriever`, `resumen`, `flashcards` y `explicacion`. La función `should_continue` inspecciona el último mensaje: si contiene `tool_calls`, redirige al nodo `tools`; en caso contrario, finaliza. Esto crea el ciclo `orquestador` → `tools` → `orquestador` que implementa el patrón ReAct [5].

Las instrucciones del sistema definen el protocolo: para preguntas libres, el modelo llama primero a `retriever` y luego a `explicacion`; para resumen y flashcards, los invoca directamente sin recuperación previa.

4.5 Gestión del Historial

La gestión del historial pasó por dos aproximaciones. La inicial mantenía el historial en memoria: se perdía al reiniciar el servidor, las sesiones largas podían superar la ventana de contexto del LLM, y en entornos multiusuario la gestión de contextos aislados requería lógica adicional.

La solución adoptada persiste cada mensaje en `chat_messages` con las columnas `rol`, `contenido` y `document_id`. Al inicio de cada invocación, `get_historial()` recupera los mensajes y los reconstruye como objetos `HumanMessage` y `AIMessage` de LangChain. Se aplica un límite sobre los mensajes más recientes para no superar la ventana de contexto, implementando una estrategia de ventana deslizante. El historial persiste entre sesiones: el estudiante puede retomar conversaciones anteriores sin perder el contexto acumulado.

4.6 Modelos de Lenguaje

El sistema fue evaluado con tres modelos que representan distintos puntos del espacio capacidad-coste-latencia.

Gemma 4 E4B es un modelo compacto de Google DeepMind ejecutado localmente mediante Ollama, con 4.5 mil millones de parámetros efectivos y una ventana de contexto de 128K tokens. Está diseñado para ejecutarse en dispositivos con recursos limitados, ofreciendo tiempos de respuesta bajos sin depender de APIs externas. Su capacidad de seguir instrucciones complejas de herramientas en sesiones largas es inferior a la del modelo mayor.

Gemma 4 31B, también ejecutado localmente mediante Ollama, es la variante de mayor tamaño de la familia Gemma 4, pensada para estaciones de trabajo con GPU. Ofrece una mejora significativa en razonamiento y seguimiento del protocolo de herramientas, con una ventana de contexto de 256K tokens, a costa de mayor exigencia de memoria y latencias de generación más elevadas.

Claude Haiku de Anthropic, accedido mediante la API de ChatAnthropic, produjo los mejores resultados en coherencia explicativa y seguimiento de instrucciones. Su arquitectura optimizada ofrece un equilibrio favorable entre calidad, velocidad y coste por token.

La abstracción `ChatModel` de LangChain permite sustituir el modelo actualizando únicamente `chat_model` en el módulo de configuración, sin modificar ninguna otra parte del código.

4.7 Streaming Asíncrono

El grafo compilado se ejecuta mediante `astream`, que emite eventos por token en lugar de esperar la finalización completa. La transmisión al cliente se realiza mediante SSE:

```
data: {"token": "El"}
data: {"token": " doc"}
data: {"token": "umento"}
```

Este diseño elimina la espera perceptible, reduce la presión sobre la memoria del servidor y simplifica el manejo de timeouts. En la aplicación, cada token recibido se concatena al estado local del componente, provocando una re-renderización progresiva.

5 Evaluación

5.1 Configuración Experimental

El sistema fue evaluado con documentos sobre temas cotidianos y de uso general, sin estar restringidos a ningún dominio técnico concreto. Configuración base: modelo *nomic-embed-text-v2-moe* con vectores de 512 dimensiones, fragmentos de 500 caracteres con solapamiento de 100, umbral de similitud 0.5 y k=5 fragmentos por consulta.

5.2 Resultados Cualitativos

Se evaluaron las cuatro funcionalidades principales del sistema: generación de flashcards, resúmenes, explicaciones de conceptos y preguntas libres sobre el contenido del documento. El sistema demostró buen rendimiento en todas ellas, con diferencias notables entre modelos. Claude Haiku produjo las respuestas más coherentes y mejor estructuradas en todas las tareas. Gemma 4 31B ofreció resultados sólidos aunque con mayor latencia, especialmente en la generación de resúmenes largos. Gemma 4 E4B fue el más rápido al ejecutarse localmente, aunque mostró dificultades ocasionales para seguir el protocolo de herramientas en preguntas libres complejas.

5.3 Limitaciones

Ventana de contexto. El sistema solo pasa al modelo los fragmentos recuperados, no el documento completo. Si la información relevante no supera el umbral de similitud o no está entre los k=5 fragmentos recuperados, el modelo simplemente no la verá y la respuesta puede quedar incompleta.

Contenido no textual. El sistema extrae únicamente el texto del PDF. Imágenes, diagramas y tablas con formato complejo son ignorados, lo que puede ser una limitación importante según el tipo de documento.

Calidad del chunking. Fragmentar por número de caracteres sin tener en cuenta la estructura del texto puede partir un concepto a la mitad, generando embeddings poco representativos que dificultan la recuperación.

Latencia local. Gemma 4 31B, al ejecutarse localmente, introduce latencias notables en tareas que generan respuestas largas, como los resúmenes completos del documento.

6 Conclusión

Este trabajo ha presentado el diseño e implementación de un sistema RAG orientado al apoyo académico desde documentos PDF. La arquitectura de tres capas desacopladas, comunicadas mediante REST y SSE, ofrece una experiencia conversacional en tiempo real.

Cambiar el diseño del agente, pasando de un grafo con router a un agente con tool calling, fue una de las decisiones más importantes del proyecto: el sistema ganó en flexibilidad y dejó de fallar ante preguntas ambiguas. De forma similar, guardar el historial en base de datos en lugar de mantenerlo en memoria fue imprescindible para que el chat funcionara de verdad, permitiendo retomar conversaciones entre sesiones.

Hay varias mejoras que quedarían pendientes para una versión futura: incorporar técnicas de re-ranking para mejorar la precisión de los fragmentos recuperados; desarrollar una estrategia de chunking que respete la estructura del documento en lugar de cortar por número de caracteres; añadir soporte para contenido visual como imágenes y diagramas dentro de los PDFs; explorar representaciones en grafo del conocimiento para capturar relaciones entre conceptos que la búsqueda vectorial no detecta; y establecer un proceso de evaluación más formal con métricas objetivas que permitan comparar el rendimiento del sistema de forma reproducible.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 9459–9474, 2020.

[2] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv:2312.10997*, 2023.

[3] D. Baidoo-Anu y L. Owusu Ansah, "Education in the Era of Generative AI," *Journal of AI*, vol. 7, no. 1, 2023.

[4] B. S. Bloom, "The 2 Sigma Problem," *Educational Researcher*, vol. 13, no. 6, pp. 4–16, 1984.

[5] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv:2210.03629*, 2022.

[6] A. Kane, "pgvector: Open-source vector similarity search for Postgres," 2023. [Online]. `https://github.com/pgvector/pgvector`

[7] LangChain, "LangGraph," 2024. [Online]. `https://github.com/langchain-ai/langgraph`

[8] A. Vaswani et al., “Attention Is All You Need,” *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.

[9] Supabase, “Supabase Documentation,” 2024. [Online]. https://supabase.com/docs

[10] S. Ramírez, “FastAPI,” 2024. [Online]. https://fastapi.tiangolo.com

[11] Ollama, “Ollama Documentation,” 2024. [Online]. https://ollama.com

[12] Nomic AI, “nomic-embed-text-v2-moe,” 2024. [Online]. https://huggingface.co/nomic-ai/nomic-embed-text-v2-moe